

Week 06 Lab: Code Documentation & AI Auditing

Software Project Documentation · 15-Week Practical Course

National University of Mongolia
Department of Computer Science
Summer Semester 2026

Topic: JSDoc/Sphinx & GitLab CI

Timebox: 90m Lecture + 90m Lab

Instructor: Ph.D. Batnyam Battulga

SPRINT GOAL & STRATEGIC VISION

Document all public API symbols of your lead project using standardized docstrings (Google/reST or JSDoc) and execute an automated Sphinx/TypeDoc build via **GitLab CI/CD**. Validate docstring quality against Chinchilla's Six Principles and perform a critical Manual vs. AI (GPT-4/Claude) audit using Bhatti's Five Principles to detect hallucinations. Clean up technical debt by converting inline comments to **GitLab Issues**.

PRODUCT BACKLOG (SPRINT USER STORIES)

TRACEABLE LITERATURE BASE

US-6.1 Public API Docstrings & Chinchilla Quality Audit

Chinchilla Ch. 7 / Bhatti Ch. 5

3 SP

"As a developer, I want to annotate all public API symbols with structured docstrings following Chinchilla's six principles so that our automated renderer produces a complete, consistent API reference."

Acceptance Criteria:

- Comprehensive Documentation:** Every public symbol has a description, parameters, return types, and possible errors documented.
- Consistent Examples:** Each docstring contains a realistic `@example` matching Chinchilla's consistent domain schema (Ch. 7, pp. 89-90).
- Internal Exclusion:** Internal helpers bear a comment explaining their exclusion from the public API.

US-6.2 Docs-as-Code CI/CD Automation (GitLab)

Chinchilla Ch. 6 (pp. 69-82)

3 SP

"As a Docs Engineer, I want to configure Sphinx/TypeDoc within GitLab CI/CD so that every push automatically builds and publishes up-to-date API documentation via GitLab Pages."

Acceptance Criteria:

- GitLab Configuration:** Valid `.gitlab-ci.yml` configured with a Sphinx/TypeDoc build job.
- Strict Compilation:** Build pipeline enforces zero warnings (e.g., using `sphinx-build -W`).
- Artifact Deployment:** Generated HTML documentation is publicly accessible via **GitLab Pages**.

US-6.3 Manual vs. AI Docstring Audit & Hallucination Log

Bhatti Ch. 5 / Chinchilla Ch. 9

4 SP

"As a Technical Writer, I want to compare manual vs. LLM-generated docstrings using Bhatti's principles so that AI hallucinations and ghost APIs are eliminated."

Acceptance Criteria:

- Diff Analysis:** Diff file comparing manual vs. AI (GPT-4/Claude) versions of the same module.
- Bhatti Audit:** Audit log utilizing the 5 principles (Explained, Concise, Clear, Usable, Trustworthy) with manual corrections applied.
- Hallucination Detection:** At least one "invented API" discovered and documented in a Use-Verify-Cite log.

US-6.4 Technical Debt Cleanup (TODO/FIXME)

Bhatti Ch. 4 (p. 79)

2 SP

"As a Quality Auditor, I want to resolve or track all inline TODO/FIXME comments as GitLab Issues so that code comments never become 'lying comments'."

Acceptance Criteria:

- Issue Creation:** Lingering TODOs/FIXMEs tracked as actionable **GitLab Issues**.
- Code Refactoring:** Remaining comments replaced with a reference format: `# TODO(#123): Refactor...`.

MAJOR PRACTICAL LAB TASK: UE-6 (120 MIN)

CORE PIPELINE WORK

Task Overview: Take a primary Public API class from your lead project and complete the four-part docstring engineering pipeline:

Phase	Action Item & Methodological Requirement	Deliverable
(a) Manual Authoring	Write complete docstrings manually. Apply Chinchilla pp. 89-90 by using a consistent domain example.	<code>manual_docstring.py</code>
(b) AI Generation & Bhatti Audit	Prompt GPT-4/Claude to generate docstrings. Evaluate in an Audit Matrix across Bhatti's 5 Principles. Identify ≥ 2 corrections.	<code>bhatti_audit_table.md</code>
(c) reST & Sphinx Integration	Merge corrected docstrings into the codebase. Integrate into Sphinx autodoc and build clean HTML without warnings.	Sphinx Build Logs & HTML
(d) Hallucination Test (Bonus)	Write an automated test script (using reflection) asserting all AI-referenced methods and types actually exist.	<code>test_ai_hallucinations.py</code>

REFERENCE CI/CD WORKFLOW: GITLAB PAGES

To deploy your Sphinx documentation automatically, use the following configuration in your `.gitlab-ci.yml` file. Notice that the output directory must be named `public` for GitLab Pages to recognize it.

```
stages:
  - deploy

pages:
  stage: deploy
  image: python:3.11
  script:
    - pip install sphinx sphinx-rtd-theme autodoc
    # -W enforces zero warnings; output must go to 'public'
    - sphinx-build -W -b html docs/source public
  artifacts:
    paths:
      - public
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

SPRINT 06 DEFINITION OF DONE (DOD) CHECKLIST

- ☐ **Sphinx / TypeDoc Build:** Green status in GitLab CI pipeline (zero warnings).
- ☐ **Live Documentation URL:** API reference website accessible via GitLab Pages.
- ☐ **Manual vs. AI Audit:** `bhatti_audit_table.md` documented with at least 2 manual AI corrections.
- ☐ **Technical Debt Integrity:** All lingering `# TODO` comments resolved or linked to GitLab Issue IDs.

AGILE CEREMONIES & REFLECTION PROMPTS

Daily Standup Prompts (3 Questions): - How many public API symbols are fully documented today? - Does the Sphinx build run cleanly in GitLab CI without warnings? - What AI hallucinations were caught by your audit?	Sprint Retrospective Questions: - Which docstring style produced the highest quality? - Which AI-generated passages required immediate manual correction? - What will we adjust in our Sprint 7 workflow?
Required Readings	- Bhatti et al.: <i>Docs for Developers</i> (Apress 2021) — Ch. 4 & Ch. 5. - Chinchilla, F.: <i>Technical Writing for Software Developers</i> (Packt 2024) — Ch. 6, 7, & 9.
Weekly Reflection Questions	- Which concept from Bhatti challenged your previous code commenting habits the most? - Which concept from Chinchilla will you apply in future software projects? - Where is the largest discrepancy between 'literature' and 'team practice'?

